

## How, What and Why Erlang?

A conventional programmer would look at Erlang as a language which tackles concurrency, resilience and flow of data in a very unique manner and this may seem unusual or difficult to some of you. But some of these features along with excellent learning material makes the language very attractive once you are in the loop with all of these aspects. OTP or Open Telecom Platform forms the most important part of this language. It is a set of libraries and a full fledged framework that provides communication protocols, static analysis tool, interpreter, compiler and a distributed database called Mnesia. The language can be compiled and run on top of a virtual machine just like Java, or it can also be used in an interactive environment like Python. The compiler, known as beam, converts the source code into a byte code, which is then converted to threaded code at the time of loading. Some of the core principles behind how the language works can be summed up with the help of below points:

### ◆ **Concurrency and Inter-Process Communication**

Concurrency and distributed computing is one of the primary facets on which the language has been built from the ground up. A major factor that helps in concurrency is the fact that all data structures in Erlang are immutable and cannot be changed, this avoids having to focus on locks, semaphores and other ways of trying to ensure the sanctity of shared data. Processes are like the primary building block that form a part of an Erlang program, and each process communicates with another using an asynchronous message passing protocol. No data is shared between any two processes and this is irrespective of the fact whether these processes are running on the same node or are part of a larger distributed cluster. This also makes it more secure and predictable, because data is shared only when there is clear and explicit communication between the two processes.

### ◆ **Pattern Matching**

Pattern matching is another feature that is inherited from the concept of functional programming. This practice promotes declarative syntax and makes working with complex data structures much easier. List Comprehensions for example, which are inspired from Lisp and other such languages also form an important part of Erlang.

### ◆ **Robustness**

Erlang has been made to ensure fault tolerance and robustness of a system.